



Thibaut Waechter

Promotion : 2024-2025

Année universitaire : 2024-2025

Soutenance de Projet de fin d'étude

eSWARM

Organisme d'accueil : iCube

300 Boulevard Sébastien Brandt

67400 Illkirch

Encadrant : Sylvain Durand

sylvain.durand@insa-strasbourg.fr

Soutenance : 05/09/2025

Plan de la présentation

Introduction

État de l'art

Description des méthodes

Description des réalisations

Résultats

Conclusion

Introduction

Contexte du stage

- **Organisme d'accueil** : ICube
- **Équipe** : RDH (Robotique, Science des Données et Santé)
- **Axes de recherche de l'équipe** :
 - Robotique médicale
 - Apprentissage et science des données
 - Systèmes complexes et mécatronique
- **Lieu du stage** : INSA Strasbourg
- **Tuteur de stage** : Mr. Sylvain Durand

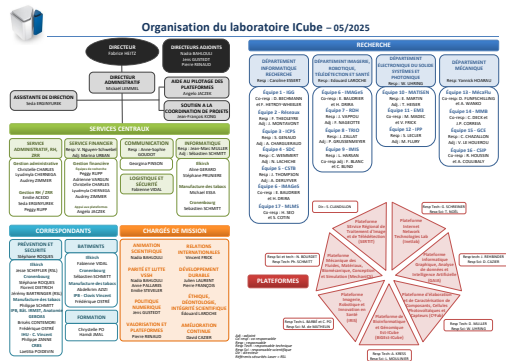


Figure 1 : Organigramme du laboratoire ICube

Objectifs : Le projet eSWARM

- Stage dans la continuité du projet **eSWARM**.
- **Objectif global** : développer un système de drones quadricoptères **modulaire** et **reconfigurable**.
- Capable de s'assembler en vol pour des missions complexes (ex : transport de charges lourdes).
- **Inspiration** : projet ModQuad [saldana_modquad_2018], un essaim de drones reconfigurable.



Figure 2 : Système ModQuad [saldana_modquad_2018].

Objectifs : Cadre du stage

- **Simplification** : Remplacement des drones par des robots terrestres holonomes.
- **Objectifs du stage** :
 1. Développer la plateforme de robots terrestres.
 2. Mettre en place une architecture logicielle distribuée (ROS2).
 3. Optimiser la commande distribuée de l'essaim.

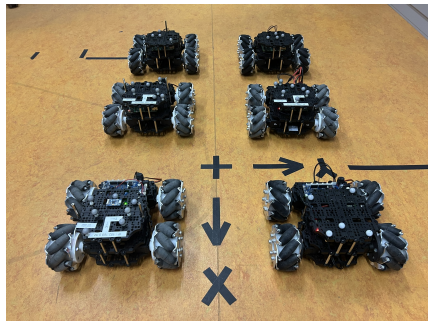


Figure 3 : Plateforme expérimentale de robots terrestres.

- **Aspect recherche :**
 - Comment optimiser la consommation énergétique des robots?
 - Comment minimiser les échanges d'informations entre les modules?
- **Approche proposée :**
 - Implémentation de paradigmes de **commande événementielle** (Event-Triggered Control).
 - Analyse des performances des algorithmes testés.

État de l'art

- **[boegeat_rapport_2023-1]** : structure d'accroche Crazyflie imprimée 3D avec aimants
- **[boegeat_rapport_2023]** : amélioration du système, documentation
- **[bonnel_modeling_2024]** : modélisation de deux quadricoptères couplés (équivalent octocoptère)



Figure 4 : UAV eSWARM

Exemples de systèmes modulaires

Systèmes non autonomes :

- **DRAGON Drone [zhao_design_2018]** :
modules articulés, modification de forme

Systèmes autonomes :

- **ModQuad [saldana_modquad_2018]** :
quadricoptères avec accrochage
magnétique
- **TRADY [sugihara_design_2023]** : aimants
commutables + ergots
- **BEATLE [sugihara_beatle_2024]** : contrôle
distribué
- **LEGION [junichiro_workshop_2025]** :
système tri-coptères

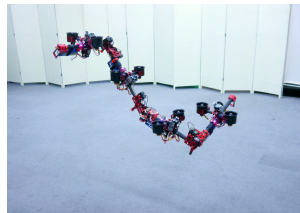


Figure 5 : DRAGON Drone



Figure 6 : BEATLE

Contrôle centralisé

- + Précision élevée
- + Calcul global simplifié
- Dépendance aux communications

Contrôle décentralisé

- + Robustesse
- Coordination complexe

Stratégie eSWARM :

1. Implémentation centralisée (simplicité)
2. Évolution vers une approche décentralisée

[jamshidpey_centralization_2024]

Algorithmes principaux :

- **Loi de consensus** [ali_practical_2018, koung_consensus-based_2020] :
 - $u_i(t) = \sum_{j=1}^n a_{ij}(t)[x_j(t) - x_i(t)]$
- **Leader-follower** [loria_leaderfollower_2016, yang_adaptive_2021] :
 - Hiérarchie avec robots leaders et followers
- **Changement de modèle** [inglett_object_2017] :
 - Détermination d'une nouvelle dynamique

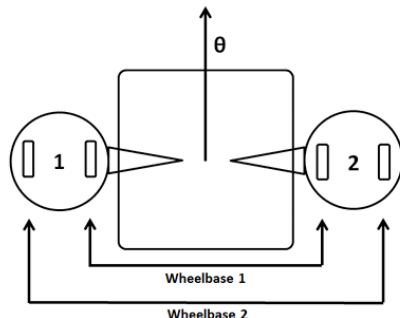


Figure 7 : Changement de modèle

Principe [astrom_event_2008] :

- Alternative au contrôle périodique
- Actions déclenchées par événements
- Réduction des ressources

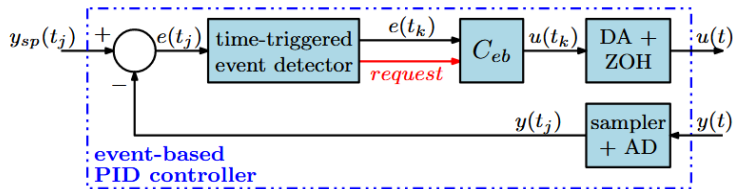


Figure 8 : Schéma de commande événementielle [durand_event-based_2015]

Commande événementielle prédictive

Objectif : réduire les communications entre modules

Principe :

- Estimation de l'état des voisins
- Prédiction basée sur modèle dynamique
- Communication uniquement si écart > seuil

Variantes de prédiction :

- **Boucle ouverte** : modèle simple, précision limitée
- **Boucle fermée** : intègre les commandes, plus précise

Avantages :

- Réduction trafic réseau
- Adaptée aux grands systèmes

Inconvénients :

- Complexité calcul
- Dépendance au modèle

- **ModQuad-DoF** : le plus proche, mais logique centralisée
- **Laboratoire DRAGON** : approche décentralisée, drones sophistiqués
- **eSWARM** : positionnement unique avec drones simples + approche distribuée

Commande événementielle :

- Domaine de recherche actif et riche
- Peu d'applications concrètes pour essais modulaires
- Opportunité d'innovation pour eSWARM

Description des méthodes

Méthode du consensus pour essaim

Algorithme retenu : Consensus distribué basé sur PID
[koug_consensus-based_2020]

Modèle intégrateur pur : $\dot{\mathbf{p}}_i = \mathbf{u}_i$ (adapté aux robots holonomes)

Commande décomposée en deux termes :

$$\mathbf{u}_i = \mathbf{u}_i^\alpha + \mathbf{u}_i^\gamma \quad (1)$$

- **Formation ($\mathbf{u}_i^\alpha$) :** Maintien des distances via PID

$$\mathbf{u}_i^\alpha = \sum_{j \in \mathcal{N}_i} \left(K_p \phi_\alpha \mathbf{n}_{ij} + K_i \int_0^t \phi_\alpha \mathbf{n}_{ij} d\tau \right) \quad (2)$$

- **Navigation ($\mathbf{u}_i^\gamma$) :** Guidage vers la cible

$$\mathbf{u}_i^\gamma = -c_{1,\gamma}(\mathbf{p}_i - \mathbf{p}_r) \quad (3)$$

Commande événementielle classique

1. Écart de distance avec les voisins

Pour chaque voisin j de i :

$$|d_{ij} - d_{ij}^*| > \delta_d = 0,05 \text{ m}$$

2. Changement du point cible global

$$\mathbf{g}^{\text{new}} \neq \mathbf{g}^{\text{old}}$$

3. Proximité ou variation significative

$$\|\mathbf{p}_i - \mathbf{p}_i^*\| < \delta_c = 0,1 \text{ m}$$

$$\text{ou } ||\|\mathbf{p}_i - \mathbf{p}_i^*\| - \|\mathbf{p}_i^{\text{old}} - \mathbf{p}_i^{*,\text{old}}\|| > \delta_t = 0,02 \text{ m}$$

Définitions

d_{ij} Distance actuelle

d_{ij}^* Distance désirée

$\mathbf{p}_i$ Position du robot i

$\mathbf{p}_i^*$ Position cible du robot i

$\mathbf{g}$ Point cible global

Seuils

δ_d Écart distance

δ_c Proximité cible

δ_t Variation position

Commande événementielle prédictive

Objectif : Limiter les échanges d'informations entre robots

Formule de prédiction :

$$\hat{\mathbf{p}}_i = \mathbf{p}_i^{\text{pub}} + \mathbf{v}_i \cdot (t_{\text{actuel}} - t_{\text{pub}})$$

Condition de publication :

$$\|\mathbf{p}_i - \hat{\mathbf{p}}_i\| > \delta_p$$

Avec $\delta_p = 0.3\text{cm}$

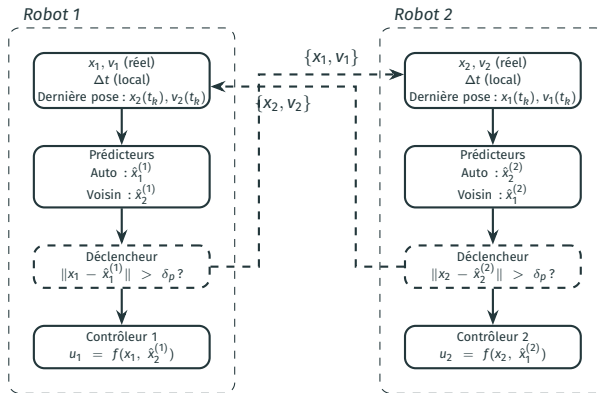
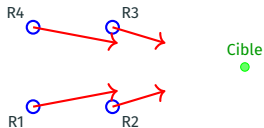


Figure 9 : Schéma de la commande événementielle prédictive

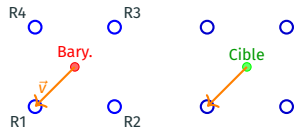
Problème : Point cible unique



Conséquences :

- Risque de collisions
- Perte de formation

Solution : Points cibles individuels



$$\mathbf{p}_r^i = \mathbf{p}_{\text{goal}} + (\mathbf{p}_i^{\text{init}} - \mathbf{p}_{\text{bary}}^{\text{init}}) \quad (4)$$

Rappel - Terme de navigation :

$$\mathbf{u}_i^\gamma = -c_{1\gamma}(\mathbf{p}_i - \mathbf{p}_r) \quad (5)$$

Objectif :

- Adapter dynamiquement l'attraction vers la cible

Principe :

- Surveillance continue des ratios de distance
- Réduction progressive selon la gravité

Trois niveaux d'adaptation :

1. **Cas extrêmes** : $c_1^\gamma \approx 0$
2. **Formation correcte** : $c_1^\gamma = 0.2$
3. **Formation imparfaite** : réduction progressive

Description des réalisations

ROS 2 : Fonctionnement

- **ROS** (Robot Operating System) est un framework qui facilite la communication entre les composants d'un robot (capteurs, actionneurs, etc.).
- L'architecture est basée sur des **nœuds** (programmes) qui communiquent via des **topics** (canaux).
- Ce système utilise un modèle **publisher-subscriber** : un nœud publie des données sur un topic, et un autre y souscrit pour les recevoir.

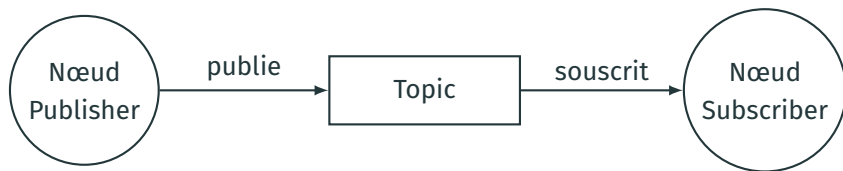


Figure 10 : Communication publisher-subscriber via un topic.

Le robot TurtleBot Mecanum

- Plateforme mobile holonome grâce aux roues Mecanum :
 - Roues à galets inclinés permettant translations dans toutes les directions et rotation sur place.
 - Combinaison des vitesses des 4 roues pour générer v_x , v_y et ω .
- Composition matérielle : Raspberry Pi 4, carte OpenCR, moteurs Dynamixel, batterie LiPo 3S.
- Communication et contrôle : micro-ROS (USB/série) et Wi-Fi

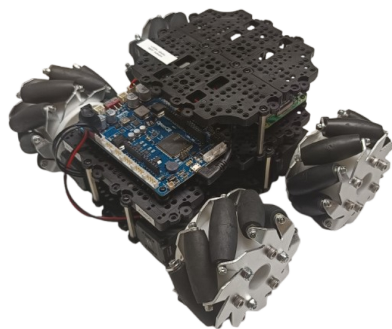


Figure 11 : Robot TurtleBot Mecanum.

Table 1 : Configurations matérielles et logicielles des robots

Robot	Ubuntu	RAM (Go)	CPU	Stockage (Go)
Aramis	22.04 Desktop	4	64-bit quad-core	32
Athos	22.04 Desktop	4	64-bit quad-core	32
Porthos	22.04 Desktop	4	64-bit quad-core	32
d'Artagnan	22.04 Server	2	64-bit quad-core	16
Edmon	22.04 Server	2	64-bit quad-core	16
Mercedes	22.04 Server	2	64-bit quad-core	16

Système de Motion Capture (OptiTrack)

- Suivi d'objets en temps réel par capture de mouvement.
- Utilise des caméras infrarouges et des marqueurs réfléchissants sur les robots.
- Le logiciel Motive calcule la pose (position et orientation) des objets (appelés *Rigid Bodies*).
- Communication avec ROS 2 via le protocole VRPN pour une localisation précise.

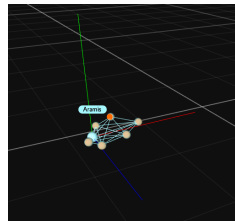
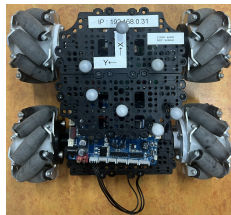


Figure 12 : Marqueurs sur un robot et son *Rigid Body* dans Motive.

Organisation simplifiée du système ROS2

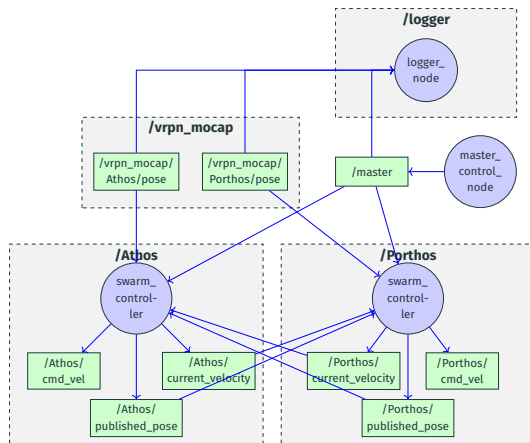


Figure 13 : Organisation simplifiée du système ROS2 pour la méthode prédictive

Résultats

Protocole d'expérience

La méthode de test est la même pour toutes les expériences :

- 4 robots en formation carrée (Figure 14).
- Les robots voisins partagent un côté (pas de diagonales).
- La trajectoire à suivre est identique pour chaque test (un carré).

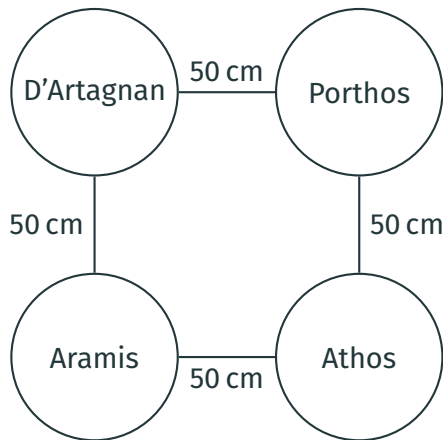
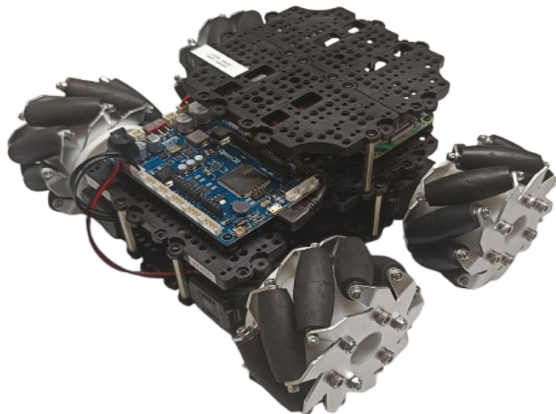


Figure 14 : Graphe de la formation

Exemple vidéo de l'essaim



Exemples de résultats

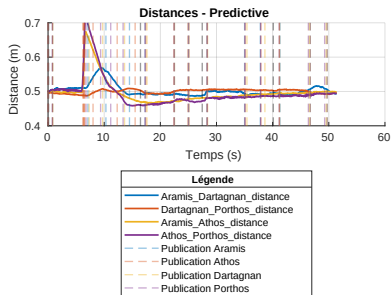


Figure 15 : Evolution des inter-distances entre robots

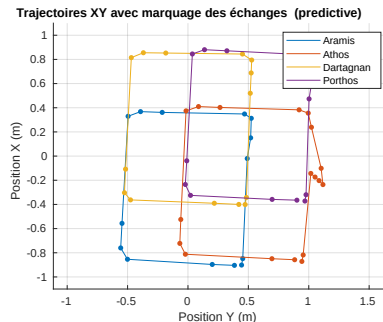


Figure 16 : Marquage des commandes

Exemples de résultats

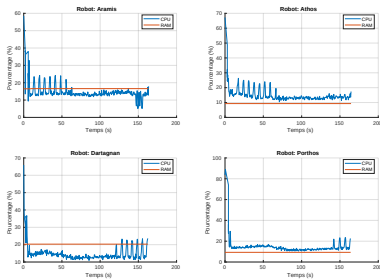


Figure 17 : Utilisation CPU et RAM
(méthode prédictive)

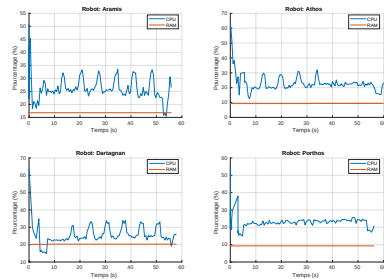


Figure 18 : Utilisation CPU et RAM
(méthode classique)

Tableau récapitulatif des performances

Table 2 : Tableau récapitulatif des performances par méthode

Critère	Classique	Événementielle	Prédictive
Temps de parcours [s]	48	39	56
Temps de convergence [s]	13	26	15
Nombre total de commandes calculées	1932	442	2187
Calculs de commande par seconde	≈ 40.3	≈ 11.3	≈ 39.1
Nombre total d'échanges de données	966	780	110
Échanges de données par seconde	≈ 20.1	20	≈ 1.96
Utilisation CPU [%]	≈ 20	≈ 20	≈ 15
Utilisation RAM [Mo]	≈ 470	≈ 470	≈ 470

Conclusion

- Mise en place de la plateforme expérimentale sous ROS2
- Adaptation d'un algorithme de contrôle d'essaim pour robots holonomes
- Optimisation de la commande via deux approches événementielles :
 - **Classique** : réduction des calculs de commande ($\approx 5x$)
 - **Prédictive** : réduction des communications ($\approx 10x$)
- Objectifs du cahier des charges atteints

- Validation des algorithmes sur des drones quadricoptères
- Étude de la consommation énergétique des robots
- Implémentation dans un environnement sans motion capture (odométrie, Lidar)
- Ajout de l'évitement d'obstacles
- Détection automatique des voisins
- Changement de formation en temps réel

Merci pour votre attention

Questions ?

Annexes

- Données techniques supplémentaires
- Schémas détaillés
- Code source
- Références bibliographiques

Annexe : Exemples de systèmes modulaires

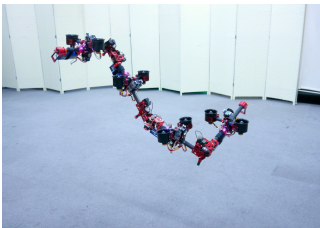


Figure 19 : DRAGON Drone

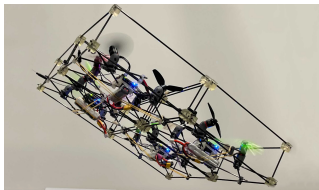


Figure 20 : ModQuad



Figure 21 : BEATLE

Annexe : Exemples de systèmes modulaires

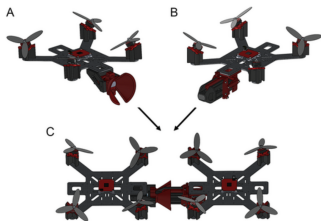
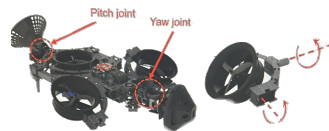


Figure 22 : Système TRADY



Figure 23 : Système BEATLE



**Figure 24 : Système
LEGION**

Détail des formules - Norme sigma

Norme sigma : Fonction de normalisation pour éviter les singularités

$$\|\mathbf{z}\|_{\sigma} = \frac{1}{\epsilon} \left(\sqrt{1 + \epsilon \|\mathbf{z}\|^2} - 1 \right) \quad (6)$$

Gradient sigma : Version normalisée du vecteur

$$\sigma_{\epsilon}(\mathbf{z}) = \frac{\mathbf{z}}{1 + \epsilon \sigma_{\text{norm}}(\mathbf{z})} \quad (7)$$

Paramètres :

- $\epsilon = 0.1$: paramètre de régularisation
- Évite les divisions par zéro quand $\|\mathbf{z}\| \rightarrow 0$

Détail des formules - Fonction de connectivité

Fonction ρ_h : Assure la connectivité entre robots

$$\rho_h(s) = \begin{cases} 1 & \text{si } 0 \leq s \leq h \\ \frac{1}{2} \left(1 + \cos \left(\frac{\pi(s-h)}{1-h} \right) \right) & \text{si } h < s \leq 1 \\ 0 & \text{sinon} \end{cases} \quad (8)$$

Fonction σ_1 : Saturation douce

$$\sigma_1(s) = \frac{s}{\sqrt{1+s^2}} \quad (9)$$

Paramètres :

- $h = 0.2$: seuil de connectivité maximale
- Transition douce vers zéro pour éviter les déconnexions brusques

Détail des formules - Terme de formation u_i^α

Fonction potentielle ϕ_α : Force d'attraction/répulsion

$$\phi_\alpha(s, d) = \frac{1}{2}\rho_h \left(\frac{s}{\sigma_{\text{norm}}(c)} \right) [(a + b)\sigma_1(s - \sigma_{\text{norm}}(d) + e) + (a - b)] \quad (10)$$

Vecteur unitaire entre robots :

$$\mathbf{n}_{ij} = \sigma_\epsilon(\mathbf{p}_j - \mathbf{p}_i) \quad (11)$$

Paramètres :

- $a = b = 1$: coefficients d'attraction/répulsion
- $e = \frac{|a-b|}{\sqrt{4ab}} = 0$: paramètre d'asymétrie
- $c = 5$: portée d'interaction entre robots
- d : distance désirée entre robots

Terme de formation complet :

$$\mathbf{u}_i^\alpha = \sum_{j \in \mathcal{N}_i} \left[K_p \phi_\alpha(\|\mathbf{p}_j - \mathbf{p}_i\|_\sigma, d_{ij}) \mathbf{n}_{ij} + K_i \int_0^t \phi_\alpha \mathbf{n}_{ij} d\tau + K_d \frac{d}{dt}(\phi_\alpha \mathbf{n}_{ij}) \right] \quad (12)$$

Gains PID :

- $K_p = 0.2$: gain proportionnel (réaction immédiate à l'erreur)
- $K_i = 0.05$: gain intégral (élimination erreur statique)
- $K_d = 0.0$: gain dérivé (amortissement, désactivé)

Rôle : Maintien des distances désirées d_{ij} entre robots voisins

Détail des formules - Terme de navigation u_i^γ

Guidage vers la cible :

$$u_i^\gamma = -c_{1,\gamma}(\mathbf{p}_i - \mathbf{p}_r) \quad (13)$$

Adaptation dynamique du gain :

$$c_{1,\gamma} = f(\text{proximité, angles, mode}) \quad (14)$$

Modes de fonctionnement :

- **Mode normal :** $c_{1,\gamma} = 0.25$
- **Mode rotation :** $c_{1,\gamma} = 0.2$
- **Adaptation :** Réduction si formation non respectée

Lissage temporel : $\alpha = 0.8$ pour éviter les transitions brusques

Détail des formules - Évitement d'obstacles

Terme d'évitement $\mathbf{u}_i^\beta$:

$$\mathbf{u}_i^\beta = c_{1,\beta} \sum_{k \in \mathcal{O}_i} \phi_\beta(\|\mathbf{p}_{ik} - \mathbf{p}_i\|_\sigma, d_\beta) \mathbf{n}_{ik} \quad (15)$$

Fonction répulsive :

$$\phi_\beta(s, d_\beta) = \rho_h \left(\frac{s}{d_\beta} \right) (\sigma_1(s - d_\beta) - 1) \quad (16)$$

Facteur de pondération :

$$b_{ik} = \rho_h \left(\frac{\|\mathbf{p}_{ik} - \mathbf{p}_i\|_\sigma}{d_\beta} \right) \quad (17)$$

Paramètres :

- $c_{1,\beta} = 0.3$: gain d'évitement
- d_β : distance de sécurité aux obstacles

Détail des formules - Saturation des commandes

Fonction de saturation :

$$\text{sat}(\mathbf{u}_i) = \begin{cases} u_{i,\min} & \text{si } u_i < u_{i,\min} \\ u_i & \text{si } u_{i,\min} \leq u_i \leq u_{i,\max} \\ u_{i,\max} & \text{si } u_i > u_{i,\max} \end{cases} \quad (18)$$

Limites physiques :

- $u_{i,\min} = -0.04$ m/s : vitesse minimale
- $u_{i,\max} = 0.04$ m/s : vitesse maximale

Commande finale saturée :

$$\mathbf{u}_i^{\text{final}} = \text{sat}(\mathbf{u}_i^{\alpha} + \mathbf{u}_i^{\gamma} + \mathbf{u}_i^{\beta}) \quad (19)$$

Détail des formules - Adaptation de c_1^γ

Conditions critiques (arrêt quasi-total) :

$$r_{\min} < 0.3 \text{ ou } r_{\max} > 2.2 \Rightarrow c_1^\gamma \approx 0 \quad (20)$$

Conditions pour valeur nominale :

$$(1 - \tau_d) \leq r_{\min} \text{ et } r_{\max} \leq (1 + \tau_d) \quad (21)$$

$$|\theta_{\text{erreur},j}| \leq \tau_\theta \quad \forall j \quad (22)$$

Réduction progressive :

$$c_1^\gamma = c_{1,\text{base}}^\gamma \cdot \min(f_d, f_\theta) \quad (23)$$

Paramètres : $\tau_d = \pm 15\%$, $\tau_\theta = \pm 15$

Détail des formules - Facteurs de réduction c_1^γ

Facteur distance :

$$f_d = f_{\text{proximité}} \cdot f_{\text{séparation}} \cdot f_{d,\text{add}} \quad (24)$$

Facteur angle :

$$f_\theta = f_{\text{erreur angle}} \cdot f_{\theta,\text{add}} \quad (25)$$

Réductions exponentielles :

$$f_{\text{proximité}} = \begin{cases} e^{-k_{\min} \cdot \delta_{\min}} & \text{si } r_{\min} < s_{\min} \\ 1 & \text{sinon} \end{cases} \quad (26)$$

Paramètres empiriques : $s_{\min} = 0.90$, $s_{\max} = 1.10$, $k_{\min} = k_{\max} = 0.4$